PROJETO DE JOGO / 01

Chees mais mais

Documento de conceito e plano de desenvolvimento • Versão 0.2 • 22/10/2026

Um roguelike tático de xadrez em um tabuleiro que começa em 5 × 5 e cresce até 8 × 5. O jogador começa com uma peça, escolhe melhorias e forma um pequeno exército para enfrentar rodadas de inimigos cada vez mais fortes. A cada cinco rodadas, um rei inimigo precisa ser derrotado por último.

Objetivo deste documento

Organizar a ideia para que o grupo consiga discutir as regras, dividir o trabalho e construir o jogo em etapas. Este material é um planejamento: não implementa o jogo nem fixa a lista de aprimoramentos que o grupo enviará depois.

Pilar	Como aparece no jogo
Identidade das peças	Peão, torre, bispo, cavalo e rei mantêm movimentos próprios; melhorias criam novas possibilidades.
Escolhas a cada rodada	Selecionar uma melhoria entre cartas sorteadas ou, nas rodadas previstas, recrutar uma peça.
Dificuldade acumulada	Dois contadores controlam a quantidade de inimigos e o orçamento de melhorias.
Leitura do combate	Tabuleiro no centro, exércitos nas laterais e capturas acima e abaixo.

Como interpretar as regras

Definido: corresponde ao pedido do grupo. Proposta: sugestão para o primeiro protótipo, sujeita a aprovação. Pendente: escolha que ainda precisa ser discutida. Essas diferenças aparecem ao longo do documento.

Requisitos do trabalho

Linguagem C + raylib; matriz para o tabuleiro; listas encadeadas para as peças; leitura e escrita de arquivo para o recorde; Makefile com suporte a Windows, Linux e macOS. Sprites e catálogo definitivo de melhorias virão depois.

Roteiro de leitura

02 Ciclo de jogo • 03 Expansão • 04 Peças e cartas • 05 Inimigos e chefes • 06 Interface e paleta • 07 Decisões • 08 Estruturas • 09 Módulos • 10 Dependências • 11 Desenvolvimento • 12 Validação.

02 • Ciclo e posicionamento

Ordem de preparação

1. Atualizar a largura do tabuleiro. 2. Oferecer as cartas e aplicar a escolha. 3. Colocar somente as peças novas ainda fora do tabuleiro. 4. Sortear e posicionar os inimigos. 5. Iniciar o combate. A expansão ocorre antes de oferecer recrutamento para que a capacidade já esteja atualizada.

Regra adicionada pelo grupo

Após escolher o aprimoramento, o jogador posiciona cada peça nova em uma casa livre das suas duas primeiras fileiras. O aliado inicial também passa por essa etapa. Peças já posicionadas não podem escolher outra casa inicial nas rodadas seguintes.

Interpretação explícita de posição fixa

A casa escolhida fica registrada como a posição inicial da peça até o final da run. Durante o combate, ela pode mover-se normalmente segundo suas regras. Proposta operacional: ao preparar outra rodada, sobreviventes voltam às suas casas iniciais registradas. Isso evita interpretar a regra como uma peça imóvel durante toda a partida; confirmar essa interpretação com o grupo.

Orientação e casas ocupadas

Proposta: na matriz desenhada de cima para baixo, as duas fileiras aliadas são as linhas 3 e 4, pois a altura é sempre 5. As demais linhas recebem inimigos. A casa inicial deve estar vazia e não pode ser a origem reservada de outro sobrevivente. Clique fora da área não consome a colocação.

Dados necessários

Adicionar à peça linhaInicial, colunaInicial e um estado de posicionamento pendente. Preservar essas coordenadas ao mover no combate. Ao capturar uma unidade, remover sua reserva. Antes de recrutar, verificar se existe uma casa inicial disponível; não basta haver espaço em qualquer parte do tabuleiro.

Cartas e resultado

Três opções comuns, cinco em chefes; nas rodadas pares uma opção de recrutamento. Recrutar abre três cartas de tipos iniciais e a unidade escolhida entra na fila de posicionamento. Ao vencer, avançar a fase; ao perder, registrar o recorde. Os detalhes de vitória, derrota e turno são fechados antes do motor.

03 • Expansão do tabuleiro

Calendário definido

Rodadas 1–5: 5 × 5 (25 casas). Rodadas 6–10: 6 × 5 (30 casas). Rodadas 11–15: 7 × 5 (35 casas). Da rodada 16 em diante: 8 × 5 (40 casas). O chefe é enfrentado antes de ganhar a coluna seguinte.

Cálculo e matriz

Usar larguraAtiva = min(8, 5 + (fase - 1) / 5), com divisão inteira e fase iniciando em 1. Reservar Peca *tabuleiro[5][8] e usar somente as colunas ativas. A altura permanece cinco. Uma nova run retorna a 5 × 5.

Expansão e posições fixas

Proposta: acrescentar cada coluna à direita, sem deslocar as coordenadas das origens já escolhidas. As duas casas da coluna nova dentro das fileiras aliadas passam a aceitar novas peças. Sobreviventes conservam suas origens; somente unidades novas escolhem posições.

Capacidade de recrutamento

A área de colocação contém 10, 12, 14 ou 16 casas, conforme a largura. Cada sobrevivente reserva uma origem. Proposta: não oferecer recrutamento quando todas as origens estiverem reservadas; indicar o motivo. A expansão aumenta a capacidade, mas ela continua finita.

Capacidade inimiga

Depois de colocar os aliados, gerar inimigos sem sobrepor casas. Definir um teto também para a área inimiga; ao não caber mais uma peça, tentar adicionar melhoria válida. Se não houver alternativa, manter a dificuldade, sem repetir sorteios indefinidamente.

Verificação

Testar fases 1, 5, 6, 10, 11, 15, 16, 20 e 21; nenhuma coluna deve abrir antes da hora. Conferir cliques e movimentos nas novas colunas, manutenção das origens e reinício em 5 × 5.

PROJETO DE JOGO / 04

Peças e aprimoramentos

O catálogo final de efeitos será fornecido pelo grupo. A estrutura deve permitir acrescentá-los sem reescrever o combate.

Peça inicial	Movimento e ação base
Peão	Uma casa para frente; duas no primeiro movimento, se o caminho estiver livre. Captura uma casa na diagonal para frente. O pedido menciona o primeiro movimento no jogo; confirmar como isso vale para peões recrutados.
Torre	Qualquer distância na horizontal ou vertical, sem atravessar outras peças.
Bispo	Qualquer distância na diagonal, sem atravessar outras peças.
Cavalo	Salta para a ponta do L: duas casas em um eixo e uma no outro. Pode saltar obstáculos.
Rei	Uma casa em qualquer direção. Ao selecionar a casa de um aliado alcançável, permanece onde está e concede a ele duas ações consecutivas na próxima jogada.

Melhorias do jogador: por tipo

Definido: sortear melhorias apenas para tipos de peça que o jogador possui; uma melhoria de peão beneficia todos os seus peões. Não repetir aprimoramentos. Proposta: guardar as melhorias por tipo durante a partida; peças recrutadas depois herdam as melhorias do seu tipo, mesmo se o grupo perder e recuperar esse tipo.

Geração das cartas

Montar pares válidos de tipo + ID de melhoria; remover pares já adquiridos e incompatíveis; sortear opções sem reposição. A regra de não repetição deve valer tanto para a mão exibida quanto para melhorias já adquiridas. Confirmar se um efeito compartilhado pode existir em tipos diferentes.

Proposta de recrutamento: sortear três tipos distintos entre os cinco iniciais. O jogador escolhe um e pode ter várias unidades do mesmo tipo. Verificar espaço disponível antes de oferecer essa carta.

Quando faltarem opções

Proposta: exibir apenas cartas válidas, mesmo que sejam menos de três ou cinco. Se não houver nenhuma, permitir avançar sem melhoria, com mensagem clara. Definir depois se haverá outra recompensa. Nunca repetir efeitos para preencher a tela nem insistir em sorteios sem fim.

Contrato do catálogo futuro

Cada melhoria precisa de: ID estável, nome, descrição, tipos compatíveis, efeito, momento de ativação, ícone e incompatibilidades. Separar melhorias do jogador por tipo e melhorias inimigas por unidade. Nenhuma lista definitiva de poderes é criada neste plano.

PROJETO DE JOGO / 05

Inimigos e chefes

A composição é refeita a cada rodada. Os contadores acumulam dificuldade, sem exigir que as mesmas peças sobrevivam entre fases.

Fases	Tipos liberados • distribuição ainda provisória
1 e 2	Somente peões.
3 e 4	Peões, bispos e torres.
5 a 9	Peões, bispos, torres e cavalos. Chefe na fase 5.
10 em diante	Adicionar rainhas, sempre sem melhorias. Chefes nas fases 10, 15, 20 etc.

Proposta para testar: pesos relativos de sorteio: peão 6, bispo 2, torre 2, cavalo 2 e rainha 1. Considerar apenas os tipos liberados. A entrada de um tipo libera sua chance de aparecer; não obriga sua presença. O rei fica reservado ao chefe. Esses pesos não são balanceamento definitivo.

Algoritmo de preparação proposto

1. A partir da rodada 2, sortear um booleano com chances iniciais iguais: true incrementa melhoriasInimigas; false incrementa pecasInimigas.
2. Sortear novamente todos os tipos permitidos, sem comparar com a rodada anterior: repetições por acaso são válidas.
3. Em rodada múltipla de cinco, escolher uma unidade e substituí-la por um rei sem melhorias. A quantidade total permanece igual.
4. Distribuir o orçamento de melhorias em pares válidos de unidade + efeito, sem repetir o mesmo efeito na mesma unidade. Excluir rainhas e rei chefe.
5. Posicionar todas as peças em casas vazias respeitando as regras de preparação.

Exemplo: rodada 1 = 3 peças e 0 melhorias; sorteio true na 2 = 3 peças e 1 melhoria; false na 3 = 4 peças e 1 melhoria. O orçamento é redistribuído do zero em cada composição, sem somar efeitos antigos novamente.

Proteções contra estados impossíveis

Propostas: se acabarem os pares válidos, manter o orçamento excedente no contador e aplicar apenas o que couber; exibir a quantidade aplicada. Se não houver espaço para mais inimigos, tentar aumentar melhorias. Se nenhuma alternativa couber, manter a dificuldade e encerrar o sorteio. Estabelecer depois um teto de unidades e regras de posicionamento.

Regra do chefe

A cada cinco rodadas, oferecer cinco cartas e impedir a captura do rei inimigo enquanto existir outro inimigo vivo. A proteção precisa existir na validação da captura, inclusive para futuros efeitos indiretos. O rei se torna capturável quando for o último; rainhas e rei chefe nunca recebem melhorias.

PROJETO DE JOGO / 06

Interface planejada

Diagrama do tabuleiro inicial 5 × 5. Direção visual definida: preto e branco com detalhes vermelhos.



Área	Informações e interação
Centro	Matriz inicial 5 × 5, expansível até 8 × 5; peça selecionada; casas de movimento e captura; indicação de turno e fase; bloqueio visível do rei chefe.
Esquerda / direita	Aliados / inimigos ativos, cada unidade com identificador e ícones de suas melhorias. Selecionar a unidade deve destacar sua casa.
Acima / abaixo	Acima: aliados que o inimigo capturou. Abaixo: inimigos que o jogador capturou. Separar o histórico da ocupação atual do tabuleiro.
Tela de cartas	Nome, tipo afetado e descrição de cada opção; três cartas comuns, cinco em chefe e três na escolha de recrutamento.

Protótipo com recursos simples

Usar formas e letras temporárias (P, T, B, C, R e D para rainha), cores de equipe e seleção por mouse. Não depender só de cor: mostrar rótulos e descrição dos ícones. Planejar rolagem nas listas laterais e nas capturas quando o conteúdo crescer.

Paleta definida: preto e branco como base; vermelho nos destaques de seleção, captura e chefe. Usar contornos, rótulos e padrões para distinguir equipes e ações, sem depender só da cor. Redimensionar e centralizar o tabuleiro ativo quando a largura aumentar.

07 • Decisões antes de programar

Fechar primeiro

Definir uma ação por lado ou uma ação por peça; quando a IA responde; condição de vitória e derrota; regras de xeque; passe e ciclos sem progresso. Escrever exemplos de um turno completo antes de dividir a implementação.

Posicionamento: regra e interpretação

Definido: novas unidades são colocadas após a carta, nas duas primeiras fileiras, sem escolher outra origem durante a run. Confirmar a interpretação da página 2: movimento normal no combate e retorno dos sobreviventes à origem entre fases. Se o grupo preferir manter a posição final do combate, essa única regra de transição deve ser ajustada antes da etapa 8 do roteiro.

Rei aliado

Proposta: toque adjacente consome ação e concede duas ações consecutivas quando o alvo agir novamente; bônus não acumula. Definir interação com outros reis e limites para impedir cadeias infinitas.

Peão

Definir promoção ao atingir a última fileira e tratamento das melhorias após promoção. Decidir se cada peão recrutado tem direito ao avanço duplo e se esse direito persiste entre fases. Não reiniciar esse estado silenciosamente ao restaurar a origem.

Progressão e limite

Aprovar pesos inimigos, regras de melhoria esgotada e limites de população. Confirmar herança de melhorias para unidades recrutadas, permanência das perdas e comportamento quando não houver origem livre.

Recorde

Escolher maior fase alcançada ou concluída. Proposta: maior fase alcançada em podio.txt. Continuar uma partida exige um salvamento separado. O roteiro das páginas 11 a 13 substitui as etapas genéricas da versão anterior.

PROJETO DE JOGO / 08

Estruturas de dados em C

Modelo sugerido para cumprir os requisitos acadêmicos e evitar duas versões conflitantes do estado do jogo.

Estrutura	Responsabilidade
Matriz do tabuleiro	Peca *tabuleiro[5][8]; armazenar altura = 5 e larguraAtiva = 5 a 8. Índices na ordem [linha][coluna]; cada casa guarda NULL ou um ponteiro para uma peça ativa. A matriz organiza posições, mas não libera a memória das peças.
Lista de aliados e inimigos	Listas simplesmente encadeadas são donas das peças ativas. Cada nó guarda ID, tipo, equipe, posição, estado do primeiro movimento e efeitos temporários; inimigos também guardam seus IDs de melhorias.
Listas de capturas	Listas encadeadas com registros de captura: tipo, equipe, fase e IDs dos efeitos, se necessários para a interface. Guardar cópias dos dados, sem ponteiros para peças já liberadas.
Melhorias aliadas por tipo	Coleção de IDs adquiridos para cada tipo de peça. Consultada por todas as unidades daquele tipo. Não duplicar a mesma coleção dentro de cada aliado.
Estado da partida	Fase, turno, tela atual, seleção por ID, pecasInimigas, melhoriasInimigas, semente aleatória e indicação de chefe. Separar dados do recorde dos dados da tentativa.

Captura como uma operação consistente

Validar a ação e a proteção do chefe; criar o registro da captura; retirar a vítima da matriz e de sua lista; limpar seleções ou referências à vítima; liberar sua memória; mover o atacante e atualizar sua posição. Se a alocação do registro falhar, tratar o erro sem deixar o tabuleiro parcialmente atualizado.

Invariantes que o grupo deve preservar

Cada peça ativa aparece uma vez em sua lista e uma vez na matriz; posição na peça e casa da matriz concordam; nenhuma casa tem duas unidades; peça capturada não permanece ativa; toda memória alocada tem um responsável por liberá-la. Ao sortear nova rodada, limpar inimigos anteriores antes de criar os novos.

Arquivo de recorde: sugestão de formato

O pedido cita podio.tsx. Recomenda-se confirmar a intenção e usar podio.txt: arquivo simples com versão e maior_fase_alcancada. Em C, ler e escrever por stdio.h; validar números, tratar arquivo ausente ou inválido e informar falha de gravação. Não gravar ponteiros ou estruturas cruas de memória.

Proposta para o histórico visual: capturas acumuladas da tentativa, com a fase indicada. Limpar listas e coleções ao iniciar outra tentativa; manter o arquivo do recorde.

Organização e compilação

Separar regras, apresentação e persistência para permitir trabalho simultâneo entre colegas sem misturar responsabilidades.

Módulo sugerido	Conteúdo
main.c / jogo.c e .h	Inicialização da raylib, loop e máquina de estados: menu, escolha inicial, cartas, preparação, combate e resultado.
tabuleiro.c / pecas.c	Matriz, listas, criação e remoção; geração e validação de movimentos; captura. Funções de regra sem chamadas de desenho.
melhorias.c	Catálogo por ID, elegibilidade, sorteio sem repetição e aplicação de efeitos nos momentos definidos.
rodadas.c / ia.c	Contadores, tipos por fase, montagem dos inimigos, chefes e escolha de ações legais da IA.
interface.c / recursos.c	Tabuleiro, painéis, cartas, entrada e carregamento dos futuros sprites. A interface pede ações ao motor, sem editar posições diretamente.
persistencia.c	Carregar e atualizar o recorde; testar ausência, conteúdo inválido e erro de escrita.

Organização proposta: src/ para fontes; include/ para cabeçalhos; assets/ para sprites e ícones; tests/ para verificações do motor; docs/ para decisões; build/ para objetos; bin/ para executáveis. A definição dos contratos de funções vem antes da implementação separada por colegas.

Makefile para três sistemas

Planejar alvos make (compilar), make run (executar), make clean (limpar gerados) e make test (testar regras sem interface). Permitir configurar compilador, opções e caminho da raylib; compilar e vincular localmente em cada sistema. Um único executável não atende aos três sistemas.

Windows: documentar um ambiente consistente, como MSYS2/MinGW com GNU Make e raylib da mesma arquitetura. Linux: documentar compilador C, GNU Make e instalação da raylib. macOS: documentar Clang, Make e raylib. Validar os parâmetros de vínculo em cada máquina; não presumir que as bibliotecas do Windows servem nos demais.

Caminhos e documentação

Definir o diretório de trabalho de make run, o caminho dos assets e uma pasta gravável para o recorde. O alvo clean deve preservar podio.txt. No README, registrar pré-requisitos, comandos por sistema e limitações conhecidas. O menu existente em src/main.c pode ser aproveitado como ponto de entrada; a opção Continuar exigirá um salvamento de partida separado se for mantida.

Dependências e ambiente

Verificação local em 22/10/2026. As ferramentas necessárias já estavam instaladas; nenhum pacote adicional foi necessário.

Ferramenta	Estado local / função
MSYS2 / MinGW64	Ambiente Windows x86_64 em C:/msys64/mingw64. Usar ferramentas e bibliotecas da mesma arquitetura.
GCC 16.1.0	Compilador C disponível. Compilação verificada com -std=c99 -Wall -Wextra.
GNU Make 4.4.1	Disponível como mingw32-make. Executará o Makefile quando ele for implementado pelo grupo.
raylib 5.5	Cabeçalho, biblioteca e DLL presentes. O pkg-config informa 5.5.0. Não foi feita atualização de versão.
pkg-config / Git	pkg-config encontra os caminhos da raylib; Git está disponível para controle de versão.

Validação realizada neste computador

O menu existente em src/main.c compilou com sucesso. Um programa mínimo também compilou, vinculou à raylib e executou ColorToInt com resultado esperado. A janela e o funcionamento gráfico não foram testados nesta revisão. Linux e macOS ainda precisam ser verificados nas máquinas do grupo.

Preparar outras máquinas

Windows / MSYS2 MinGW64: instalar os pacotes mingw-w64-x86_64-gcc, mingw-w64-x86_64-make, mingw-w64-x86_64-raylib e mingw-w64-x86_64-pkgconf com pacman -S --needed. O comando completo está em docs/DEPENDENCIAS.md.

Linux: preparar compilador C, Make, pkg-config e a raylib, com bibliotecas gráficas e de áudio requeridas pela distribuição. Para Ubuntu/Debian, o guia complementar registra os pacotes e um caminho de compilação da raylib 5.5. macOS: instalar as Command Line Tools e, com Homebrew disponível, executar brew install raylib pkg-config. Combinar a versão usada pelo grupo; o gerenciador pode oferecer outra versão.

O que faz parte do jogo

C, biblioteca padrão e raylib atendem ao escopo atual. Python e as bibliotecas de PDF usadas para editar este documento não são dependências do jogo. Não é necessário instalá-las nas máquinas dos colegas para compilar ou jogar.

Referências oficiais e instruções completas

MSYS2: gerenciamento de pacotes • raylib: GNU/Linux • raylib: macOS. Ver comandos reproduzíveis e resultado da verificação em docs/DEPENDENCIAS.md, entregue junto ao plano.

11 • Passo a passo: fundação

1. Escrever as decisões

Entregar uma folha com turno, vitória/derrota, peão, rei e transição entre rodadas. Aceite: simular dois turnos e uma transição no papel sem dúvidas. Não começar cartas ou IA antes disso.

2. Preparar a compilação

Verificar C, raylib e Make; criar Makefile mínimo para o menu existente. Aceite: compilar, abrir e fechar uma janela. Registrar o comando no README. Validar primeiro Windows; concluir os três sistemas na etapa 16.

3. Criar os dados

Implementar tipos de peça, equipe, ID e listas encadeadas. Criar matriz máxima [5][8], largura ativa, posição atual e origem registrada. Aceite: inserir, consultar e remover unidades sem duplicações.

4. Desenhar e selecionar

Desenhar somente a área ativa 5 × 5 e converter clique em linha/coluna. Usar letras temporárias em preto e branco, com vermelho nos destaques. Aceite: cliques externos não acessam a matriz.

5. Implementar a colocação

Criar fila de unidades pendentes; aceitar somente casa livre nas duas fileiras aliadas; registrar origem e impedir troca posterior. Aceite: colocar a peça inicial e duas novas unidades sem sobreposição.

6. Implementar movimentos

Fazer peão, torre, bispo, cavalo e rei, nessa ordem. Primeiro validar limites e ocupação; depois captura e primeiro avanço do peão. Aceite: cenários legais/ilegais por peça, sem IA nem cartas.

12 • Passo a passo: combate

7. Capturar e encerrar o combate

Sincronizar matriz, lista ativa e registro de captura; liberar memória e limpar seleção. Implementar vitória e derrota aprovadas. Aceite: uma rodada com inimigos fixos termina corretamente.

8. Fazer a transição de rodada

Restaurar origens conforme a decisão aprovada; preservar sobreviventes, remover mortos e abrir colocação somente para novas peças. Aceite: três rodadas manuais sem permitir reposicionar sobreviventes.

9. Adicionar turnos e IA simples

Alternar ações pela regra aprovada; IA escolhe somente movimentos legais. Tratar ausência de movimentos. Depois integrar o toque do rei e as duas ações. Aceite: combate completo sem editar o estado no código.

10. Criar o sistema de melhorias

Definir IDs e catálogo; elegibilidade por tipo; coleção aliada por tipo e inimiga por unidade. Usar poucos efeitos de teste acordados pelo grupo. Aceite: dois peões compartilham a melhoria e ela não se repete.

11. Criar cartas e recrutamento

Sortear três cartas sem reposição; garantir recrutamento nas rodadas pares quando houver origem livre. Abrir três tipos para escolher um; enviar a unidade para a colocação. Aceite: a unidade nova recebe melhorias do tipo e a origem só é definida depois da carta.

Regra de trabalho

Em cada passo: implementar, testar os cenários descritos, revisar com um colega e integrar uma versão que compila. Não iniciar a etapa seguinte enquanto o aceite falhar. Arte final, animações e balanceamento ficam para o final.

13 • Passo a passo: progressão e entrega

12. Gerar dificuldade e inimigos

Começar com três peças normais; da fase 2 em diante incrementar um contador por sorteio. Ressortear tipos liberados e redistribuir melhorias válidas. Aceite: fases mínimas respeitadas, rainha sem efeitos, nenhum laço infinito ao esgotar opções.

13. Integrar chefes e expansão

A cada cinco fases, substituir uma unidade por rei sem melhorias, protegê-lo até ser o último e oferecer cinco cartas. Ao vencer, abrir a coluna da próxima fase antes da escolha e colocação. Aceite: testar transições 5→6, 10→11 e 15→16; manter 8 × 5 depois.

14. Gravar o recorde

Implementar leitura e escrita de podio.txt, validação do conteúdo e tratamento de arquivo ausente. Aceite: perder, fechar e abrir conserva o maior recorde; make clean não apaga o arquivo.

15. Completar a apresentação

Integrar sprites e catálogo definitivo, painéis laterais, capturas, descrições e mensagens de ação inválida. Manter preto/branco e vermelho nos destaques. Aceite: outra pessoa entende colocação, turno, proteção do chefe e melhorias sem explicação oral.

16. Testar, balancear e empacotar

Compilar e jogar em Windows, Linux e macOS; ajustar Makefile e README. Jogar runs com sementes registradas, testar memória, lotação, catálogo esgotado e reinício. Ajustar pesos e poderes somente após o fluxo funcionar.

Divisão eficiente do grupo

Uma pessoa mantém contratos e integração; outra cuida do motor/listas; outra, interface e interação. Progressão e cartas podem ser divididas depois da etapa 9, com contratos estáveis. Arte pode avançar em paralelo, mas a programação usa recursos temporários até a etapa 15.